

BriskSeed: Online History-Guided Reuse for Accelerating Dynamic Approximate Nearest Neighbor Search

Hongru Gao

Huazhong University of Science and Technology (HUST)

May 29, 2026

Talk Roadmap (7-Step Logic)

- ① What is the problem?
- ② Why it matters?
- ③ Why existing work fails?
- ④ What is the key idea?
- ⑤ What is the design?
- ⑥ What is the experimental result?
- ⑦ What is the takeaway?

1) What is the Problem?

Problem: Graph-based ANNS suffers from heavy random memory access and cache misses, which limit search efficiency. Existing acceleration techniques are mostly designed for static index snapshots and degrade under continuous updates. In dynamic settings, one-shot optimization quickly becomes stale, while frequent re-optimization introduces high maintenance overhead. Therefore, the key problem is how to sustain query acceleration continuously under updates without hurting end-to-end throughput. i.e., Need to maximize

$$\text{QPS}(R) = \frac{\sum_{r=1}^R |Q_r|}{T_s(R) + T_m(R)}$$

under a recall constraint relative to baseline.

2) Why it Matters?

- ANNS is a core primitive in RAG, recommendation, anomaly detection, and fraud detection.
- In streaming scenarios, updates arrive continuously while decision latency budgets remain strict.
- Throughput decay is not just a systems issue; it can directly translate into business risk.

Representative case: real-time payment fraud detection

- Incoming transactions are embedded and matched against a continuously updated behavioral database via ANNS.
- The index must absorb new transactions while still serving latency-critical queries under millisecond-level SLAs.
- Performance degradation across updates is unacceptable because delayed retrieval can increase fraud leakage and financial loss.

Empirical motivation from the paper:

- Cross-round top- k overlap is often high.
- Query-hit frequencies are long-tail skewed.
- Per-query stability is heterogeneous.

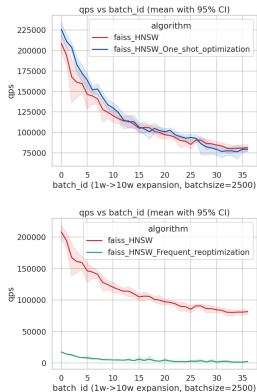
3) Why Existing Work Fails?

Static acceleration assumptions break in dynamic workloads

- Layout/quantization/hints are optimized for snapshots, not evolving indices.
- One-shot optimization gains decay as updates accumulate.
- Frequent global re-optimization is too expensive and hurts end-to-end QPS.

Dilemma:

- No maintenance $\Rightarrow$ stale guidance.
- Aggressive maintenance $\Rightarrow$ excessive overhead.



4) What Is Our Key Idea?

Turn one-shot hinting into an online reuse–validate–refresh loop.

- Reuse past successful search outcomes as compact **seeds**.
- Validate reused candidates before acceptance.
- Refresh/evict seeds online so memory tracks workload and drift.
- Keep the mechanism backend-agnostic via plugin adapters.

Intuition:

- Historical outcomes are not final answers.
- They are warm-start guidance with bounded risk.

5) What Is the Key Design? (Architecture)

Three co-designed layers

① Seed Storage Layer

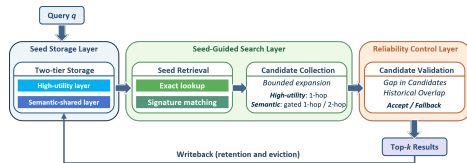
- Two-tier memory: hot-exact + semantic-shared.
- Seed record: $r = \langle \mathbf{z}, n_b, \beta, \rho \rangle$.

② Seed-Guided Search Layer

- Retrieve seed by exact key or bounded semantic probing.
- Recover candidates with one-hop / guarded two-hop expansion.

③ Reliability Control Layer

- Accept only if quality and consistency both pass.
- Otherwise immediate fallback to baseline ANN search.



5) What Is the Key Design? (Pipeline and Guarantees)

Bounded online pipeline per query

- Retrieval cost: $O(Pm)$.
- Recovery cost: $O(kM)$ or $O(kM^2)$.
- Validation + writeback: bounded w.r.t. dataset size N .

Expected latency

$$T_{\text{BriskSeed}} = T_{\text{reuse}} + (1 - p_{\text{acc}})T_b, \quad T_{\text{reuse}} = O(Pm + kM^2)$$

Benefit condition

$$T_{\text{reuse}} < p_{\text{acc}} T_b$$

Consequence: graceful degradation under drift, and speedup when reuse remains sufficiently valid.

6) What Is the Experimental Result? (Placeholder)

To be filled after experiments are finalized

- Main metrics:
 - End-to-end QPS across rounds
 - Recall@ k vs baseline
 - Maintenance overhead ratio
- Main baselines:
 - Backend native mode
 - Existing dynamic ANNS backends (same benchmark protocol)
- Ablation plan:
 - Two-tier storage vs single-tier
 - Validation on/off
 - One-hop vs guarded two-hop

[Insert plots/tables here]

7) What Is the Takeaway?

- Dynamic ANNS acceleration is an **efficiency-maintenance** problem, not only a search-speed problem.
- BriskSeed turns static hinting into **online reusable seed management**.
- The design is **bounded**, **reliable**, and **backend-agnostic**.
- Practical value: better end-to-end throughput while preserving competitive recall under continuous updates.

Thank you!
Questions?